IF2110 Algoritma dan Pemrograman 2

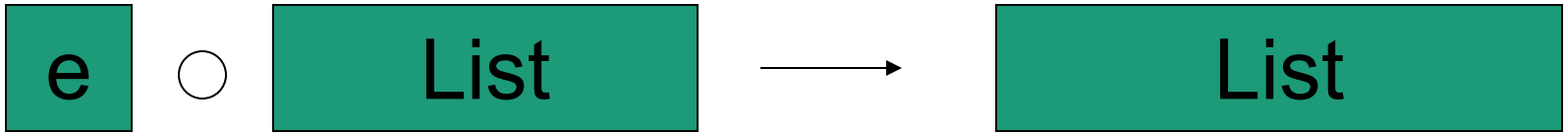
Operasi-Operasi pada List dengan Elemen Sederhana

Tim Pengajar IF2110

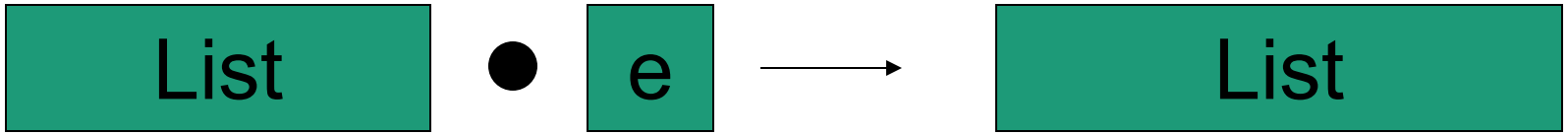
Sekolah Teknik Elektro dan Informatika

Review – List dengan elemen sederhana

- **<u>type</u>** List: [ ] atau [e o List]



- **<u>type</u>** List: [ ] atau [List • e]



TYPE LIST

DEFINISI DAN SPESIFIKASI TYPE

type List : [ ], atau [e o List]

{ Konstruktor menambahkan elemen di awal }

type List : [ ], atau [List • e]

{ Konstruktor menambahkan elemen di akhir }

{ Definisi List : sesuai dengan definisi rekursif di atas}

DEFINISI DAN SPESIFIKASI KONSTRUKTOR

Konso: elemen, List → List

{ Konso(e,L): menghasilkan sebuah list dari e dan L, dengan e sebagai elemen pertama list: e o L → L' }

Kons• : List, elemen → List

{ Kons •(L,e): menghasilkan sebuah list dari L dan e, dengan e sebagai elemen terakhir list: L • e → L' }

DEFINISI DAN SPESIFIKASI SELEKTOR

FirstElmt: List tidak kosong → elemen

{ FirstElmt(L) menghasilkan elemen pertama list L }

Tail: List tidak kosong → List

{ Tail(L) menghasilkan list L tanpa elemen pertama, mungkin kosong }

LastElmt: List tidak kosong → elemen

{ FirstElmt(L) menghasilkan elemen terakhir list L }

Head: List tidak kosong → List

{ Head (L) menghasilkan list L tanpa elemen terakhir, mungkin kosong }

DEFINISI DAN SPESIFIKASI PREDIKAT DASAR (UNTUK BASIS ANALISA REKURENS)

{ Basis 0 }

IsEmpty: List → boolean

{ isEmpty(L) true jika list kosong }

{ Basis 1 }

IsOneElmt: List → boolean

{ isOneElmt(L) true jika list L hanya mempunyai satu element }

Menghitung banyaknya elemen (nbElmt)

- **NbElmt** : List → integer
- Memanfaatkan definisi list terdiri atas FirstElmt + Tail
- Rekursif
  - Basis 0: list kosong, NbElmt = 0
  - Rekurens: nbElmt l = 1 + NbElmt (tail l)
- Realisasi

Contoh:
NbElmt([]) = 0
NbElmt([1,2,3]) = 3

```
DEFINISI DAN SPESIFIKASI
NbElmt : List → integer
{ NbElmt(L) menghasilkan banyaknya elemen list, nol
jika list kosong }

REALISASI
NbElmt(L) = if isEmpty(L) then 0        { Basis 0 }
            else 1 + NbElmt(Tail(L))     { Rekurens }
```

Mengecek keanggotaan sebuah elemen (isMember)

- Fungsi **isMember** mengecek apakah suatu elemen adalah member dari suatu list

```
IsMember : elemen, List → boolean
{ IsMember(x,L) menghasilkan true jika x adalah elemen
   dari L }
```

- Contoh:

```
IsMember(5,[]) = false
IsMember(5,[1,2,3]) = false
IsMember(5,[4,5,6]) = true
```

Rekursif

- **Basis**: jika list kosong maka nilai keluaran (output) adalah false (basis 0)
- **Rekurens:**
  Jika nilai elemen pertama (atau terakhir) dari list adalah x, maka output adalah true. Tapi jika bukan x, maka tail (atau head) harus dicek.

Fungsi isMember

```
DEFINISI DAN SPESIFIKASI (FirstElmt+Tail)
IsMember1 : elemen, List → boolean
{ IsMember1(x,L) true jika x adalah elemen list L }
REALISASI
IsMember1(x,L) : if isEmpty(L) then false  { Basis }
                 else if FirstElmt(L) = x then true
                      else IsMember1(x, Tail(L))  { Rekurens }
```

```
DEFINISI DAN SPESIFIKASI (Head+LastElmt)
IsMember2 : elemen, List → Boolean
{ IsMember2(x,L) true jika x adalah elemen list L }
REALISASI
IsMember2(x,L) : if isEmpty(L) then false  { Basis }
                 else if LastElmt(L) = x then true
                      else IsMember2(x, Head(L))  { Rekurens }
```

Menyalin (copy) list

- Proses mengambil satu persatu elemen dari List sumber dan membuat elemen baru untuk List target hasil copy

```
Copy : List → List
{ Copy(L) menyalin satu per satu elemen L untuk membentuk
   list baru hasil copy}
```

- Contoh:

```
Copy([]) = []
Copy([2,3,4]) =
[2,3,4]
```

Rekursif

- **Basis**: isEmpty → memberi list kosong [ ]
- **Rekurens**: mengambil elemen pertama list sumber kemudian memasukkannya sebagai elemen pertama list target ATAU mengambil elemen terakhir dari list sumber kemudian memasukkannya sebagai elemen terakhir list target

Fungsi copy

```
DEFINISI DAN SPESIFIKASI
Copy : List → List
{ Copy(L) menghasilkan list yang identik dengan list asal }

REALISASI (versi 1: menggunakan konso)
Copy(L) : if IsEmpty(L) then []                    { Basis }
          else Konso(FirstElmt(L), Copy(Tail(L))) { Rekurens }

REALISASI (versi 2: menggunakan konsDot)
Copy(L) : if IsEmpty(L) then []                     { Basis }
          else KonsDot(Copy(Head(L)), LastElmt(L)) { Rekurens }
```

Mengecek apakah 2 list sama atau tidak (isEqual)

- **`IsEqual`** `: 2 List → boolean`

- Contoh:

```
IsEqual([],[]) = true
IsEqual([],[a]) = false
IsEqual([a],[]) = false
IsEqual([a,b,c],[a,b,c]) = true
```

Rekursif

- **Basis**: Jika keduanya kosong maka true, jika hanya salah satu kosong maka false
- **Rekurens**: Cek elemen pertama dari kedua list dan kemudian cek tail kedua list tersebut ATAU cek elemen terakhir dari kedua list dan kemudian cek head kedua list

Fungsi isEqual

DEFINISI DAN SPESIFIKASI

IsEqual : 2 List → boolean

{ isEqual l1 l2 true jika semua elemen list l1 sama dengan l2: sama urutan dan sama nilai per elemen pada posisi yang sama }

REALISASI

```
IsEqual(L1,L2):
   depend on L1, L2
      IsEmpty(L1) and IsEmpty(L2)          : true   { Basis }
      IsEmpty(L1) and not IsEmpty(L2)      : false  { Basis }
      not IsEmpty(L1) and IsEmpty(L2)      : false  { Basis }
      not IsEmpty(L1) and not IsEmpty(L2) :          { Rekurens }
            (FirstElmt(L1) = FirstElmt(L2)) and then
                  IsEqual(Tail(L1), Tail(L2))
```

Menggabung (konkatenasi) 2 List

```
Konkat : 2 List → List
{ Konkat(L1,L2) menghasilkan konkatenasi L1 dan L2
dengan list L2 "sesudah" list L1 }
```

- Contoh:

```
Konkat([],[]) = []
Konkat([a],[b,c]) = [a,b,c]
```

- Rekursif terhadap l1
  - Basis: l1 adl. list kosong, maka output adl. l2
  - Rekurens: mengambil elemen pertama dari l1 dan menggabungkannya dengan konkatenasi antara tail(l1) dan l2.

Fungsi konkat

DEFINISI DAN SPESIFIKASI

Konkat : 2 List → List

{ Konkat(L1,L2) menghasilkan konkatenasi L1 dan L2 dengan list L2 "sesudah" list L1 }

REALISASI

```
Konkat(L1,L2) :
     if IsEmpty(L1) then L2  { Basis }
     else                    { Rekurens }
         Konso(FirstElmt(L1), Konkat(Tail(L1),L2))
```

Ambil elemen ke-N

- **`ElmtKeN`** `: integer>0, List tidak kosong → elemen`
- Prekondisi: Parameter integer bernilai besar dari 0, list tidak kosong, dan parameter integer <= jumlah elemen list
- Contoh:

  `ElmtKeN(0,[])` → tidak terdefinisi, karena dalam spek, list input harus tidak kosong dan N ≥ 1, N ≤ jumlah elemen List

  `ElmtKeN(1,[a,b,c]) = a`

- Rekurens dilakukan terhadap N (dikurangi 1) dan List (tail-nya)

ElmtKeN

DEFINISI DAN SPESIFIKASI

```
ElmtKeN : integer>0 → elemen
{ ElmtKeN(N,L) mengembalikan elemen ke-N dari list L
Prekondisi: N>0 dan N<=jumlah elemen L, list L tdk kosong }
```

REALISASI

```
ElmtKeN(N,L) : if (N = 1) then FirstElmt(L)  { Basis }
               else                          { Rekurens }
                  ElmtKeN(N-1, Tail(L))
```

Apakah X adalah elemen ke N?

IsXElmtkeN : elemen, integer>0, List tidak kosong → boolean

- Cara 1 (Rekursif):
  - Basis 1: N=1, dan kemudian mengecek apakah FirstElmt(L)=X
  - Rekurens: N dikurangi 1, X tetap dan L diambil Tail-nya
- Cara 2:
  - Menggunakan fungsi antara elmtKeN(N,L), mengecek apakah elmtKeN(N,L) = X ?
- Buatlah sebagai latihan fungsi **isXElmtkeN**.

Notasi Haskell

Review – List dengan elemen sederhana – Notasi Haskell

- Definisi type list

```
-- DEFINISI DAN SPESIFIKASI LIST

-- type List of <type_elemen>: [ ] atau [e o List]
-- Definisi type List of <type_elemen>
-- Basis: List of <type_elemen> kosong adalah list of <type_elemen>
-- Rekurens: List tidak kosong dibuat dengan menambahkan sebuah
-- elemen bertype <type_elemen> di awal sebuah list

-- type List of <type_elemen>: [ ] atau [List • e]
-- Definisi type List of <type_elemen>:
-- Basis: List of <type_elemen> kosong adalah list of <type_elemen>
-- Rekurens: List tidak kosong dibuat dengan menambahkan sebuah
-- elemen bertype <type_elemen> di akhir sebuah list
```

Review – List dengan elemen sederhana – Notasi Haskell

- Konstruktor

```
-- DEFINISI DAN SPESIFIKASI KONSTRUKTOR

konso :: <type_elemen> -> [<type_elemen>] -> [<type_elemen>]
-- konso e l menghasilkan sebuah list dari e (sebuah
-- elemen) dan l (list of elemen),
-- dengan e sebagai elemen pertama: e o l -> l'
-- REALISASI
konso e l = [e] ++ l

konsDot :: [<type_elemen>] -> <type_elemen> -> [<type_elemen>]
-- konsDot l e menghasilkan sebuah list dari l (list of
-- elemen) dan e (sebuah elemen),
-- dengan e sebagai elemen terakhir: l • e -> l'
-- REALISASI
konsDot l e = l ++ [e]
```

Review – List dengan elemen sederhana – Notasi Haskell

- Selektor

```
-- DEFINISI DAN SPESIFIKASI SELEKTOR

-- head : [<type_elemen>] -> <type_elemen>
-- head l menghasilkan elemen pertama list l, l tidak kosong

-- tail : [<type_elemen>] -> [<type_elemen>]
-- tail l menghasilkan list tanpa elemen pertama list l,
-- l tidak kosong

-- last : [<type_elemen>] -> <type_elemen>
-- last l menghasilkan elemen terakhir list l, l tidak kosong

-- init : [<type_elemen>] -> [<type_elemen>]
-- init l menghasilkan list tanpa elemen terakhir list l,
-- l tidak kosong
```

Review – List dengan elemen sederhana – Notasi Haskell

- Predikat dasar

```
-- DEFINISI DAN SPESIFIKASI PREDIKAT

isEmpty :: [<type_elemen>] -> Bool
-- isEmpty l true jika list of elemen l kosong
-- REALISASI
isEmpty l = null l

isOneElmt :: [<type_elemen>] -> Bool
-- isOneElmt l true jika list of integer l hanya
-- mempunyai satu elemen
-- REALISASI
isOneElmt l = (length l) == 1
```

Menghitung banyaknya elemen (nbElmt)

```
-- DEFINISI DAN SPESIFIKASI
nbElmt :: [<type_elemen>] -> Int
-- nbElmt l menghasilkan banyaknya elemen list, nol
-- jika list kosong

-- REALISASI
nbElmt l = if (isEmpty l) then 0       -- Basis
           else 1 + (nbElmt (tail l)) -- Rekurens
```

Menghitung banyaknya elemen (nbElmt)

```
-- DEFINISI DAN SPESIFIKASI
nbElmt :: [<type_elemen>] -> Int
-- nbElmt l menghasilkan banyaknya elemen list, nol
-- jika list kosong

-- REALISASI
nbElmt l = if (isEmpty l) then 0       -- Basis
           else 1 + (nbElmt (tail l)) -- Rekurens
```

```
-- DEFINISI DAN SPESIFIKASI, LIST OF INTEGER
nbElmt :: [Int] -> Int
-- nbElmt l menghasilkan banyaknya elemen list, nol
-- jika list kosong

-- REALISASI
nbElmt l = if (isEmpty l) then 0       -- Basis
           else 1 + (nbElmt (tail l)) -- Rekurens
```

```
-- DEFINISI DAN SPESIFIKASI, LIST OF CHARACTER
nbElmt :: [Char] -> Int
-- nbElmt l menghasilkan banyaknya elemen list, nol
-- jika list kosong

-- REALISASI
nbElmt l = if (isEmpty l) then 0       -- Basis
           else 1 + (nbElmt (tail l)) -- Rekurens
```

Fungsi isMember

```
-- Definisi FirstElmt+Tail
isMember1 :: <type_elemen> -> [<type_elemen>] -> Bool
-- isMember1 x l true jika x adalah elemen list l
-- REALISASI isMember1 menggunakan head dan tail
isMember1 x l = if (isEmpty l) then False  -- Basis
                else if (head l) == x then True
                     else (isMember1 x (tail l))  -- Rekurens
```

```
-- Definisi Head+LastElmt
isMember2 :: <type_elemen> -> [<type_elemen>] -> Bool
-- isMember2 x l true jika x adalah elemen list l
-- REALISASI isMember2 menggunakan init dan last
isMember2 x l  = if (isEmpty l) then False  -- Basis
                 else if (last l) == x then True
                      else (isMember2 x (init l))  -- Rekurens
```

Fungsi copy

```
-- DEFINISI DAN SPESIFIKASI
copy :: [<type_elemen>] -> [<type_elemen>]
-- copy l menghasilkan list yang identik dengan list asal

-- REALISASI (versi 1: menggunakan konso)
copy l = if (isEmpty l) then []                    -- Basis
         else (konso (head l) (copy (tail l)))  -- Rekurens

-- REALISASI (versi 2: menggunakan konsDot)
copy l = if (isEmpty l) then []                     -- Basis
         else (konsDot (copy (init l)) (last l)) -- Rekurens
```

Fungsi isEqual

```
-- DEFINISI DAN SPESIFIKASI
isEqual :: [<type_elemen>] -> [<type_elemen>] -> Bool
-- isEqual l1 l2 true jika semua elemen list l1 sama dengan l2:
-- sama urutan dan sama nilai per elemen pada posisi yang sama

-- REALISASI
isEqual l1 l2
   | (isEmpty l1) && (isEmpty l2) = True      -- Basis
   | (isEmpty l1) && not (isEmpty l2) = False -- Basis
   | not (isEmpty l1) && (isEmpty l2) = False -- Basis
   | not (isEmpty l1) && not (isEmpty l2) = -- Recc
       (
          (head l1)==(head l2) && (isEqual (tail l1) (tail l2))

       )
```

Fungsi konkat

```
-- DEFINISI DAN SPESIFIKASI
konkat:: [Int] -> [Int] -> [Int]
-- konkat l1 l2 menghasilkan konkatenasi l1 dan l2,
-- dengan list l2 "sesudah" list l1

-- REALISASI
konkat l1 l2 =
     if (isEmpty l1) then l2  -- Basis
     else -- Rekurens
         (konso (head l1) (konkat (tail l1) l2))
```

elmtKeN

```
-- DEFINISI DAN SPESIFIKASI
elmtKeN:: Int -> [<type_elemen>] -> <type_elemen>
-- elmtKeN n l mengembalikan elemen ke-n dari list l
-- Prekondisi: n>0 dan n<=jumlah elemen l,
-- list l tdk kosong

-- REALISASI
elmtKeN  n l = if (n == 1) then (head l)  -- Basis
               else -- Rekurens
                  (elmtKeN (n-1) (tail l))
```

Apakah X adalah elemen ke N?

```
isXElmtkeN ::
    <type_elemen> -> Int -> [<type_elemen>] -> Bool
```

- Cara 1 (Rekursif):
  - Basis 1: N=1, dan kemudian mengecek apakah FirstElmt(L)=X
  - Rekurens: N dikurangi 1, X tetap dan L diambil Tail-nya
- Cara 2:
  - Menggunakan fungsi antara elmtKeN(N,L), mengecek apakah elmtKeN(N,L) = X ?
- Buatlah sebagai latihan fungsi **isXElmtkeN**.

Latihan 1

a. Tuliskan realisasi dari fungsi **countFactorOfX** berikut ini:

```
countFactorOfX : integer, List of integer -> integer
{ countFactorOfX(n, L) mengembalikan banyaknya kemunculan bilangan yang
   merupakan faktor dari n pada L }
```

b. Tuliskan realisasi dari fungsi **delNthElmt** berikut ini:

```
delNthElmt : integer, List of integer -> List of integer
{ delNthElmt(n, L) menghilangkan elemen ke-n dari L.
   Asumsi: n lebih kecil atau sama dengan jumlah elemen L;
   L tidak kosong }
```

Beberapa Jenis List

List of Integer

- **List of integer** adalah list yang elemennya berupa integer

Elemen	Integer
List of elemen	List of integer

- Contoh
  - Konso: elemen, List → List *{untuk list of elemen}*
  - Konso: integer, List of integer → List of integer *{untuk list of integer}*

Nilai Maksimum

- Fungsi menghasilkan elemen bernilai maksimum dari list bilangan integer yang tidak kosong
- **maxlist** : List of integer -> integer
- Rekursif
  - Basis list 1 elemen: jika elemen list berjumlah satu maka ambil nilai terakhir dari list. Basis 1 digunakan karena jika list kosong maka nilai maksimum tidak terdefinisi
  - Rekurens: membandingkan nilai elemen terakhir list dengan nilai maksimum dari head list

```
DEFINISI DAN SPESIFIKASI
maxList : List of integer -> integer
{ maxList(L) mengembalikan elemen terbesar pada L
 prekondisi: list tidak kosong }
REALISASI
maxList(L) : if isOneElmt(L) then LastElmt(L) { Basis 1 }
             else { Rekurens }
                 max2(LastElmt(L), maxList(Head(L)))
```

max2(a,b) adalah fungsi yang mengirimkan nilai maksimum dari dua buah integer a dan b yg pernah dibahas sebelumnya

Penjumlahan Dua List Integer

- Fungsi untuk menjumlahkan dua list integer yang hasilnya disimpan dalam satu list integer. Asumsi: kedua list input memiliki dimensi (jumlah elemen) yang sama.
- **`listPlus`** `: 2 List of integer -> List of integer`
- Rekursif
  - Basis list kosong: Jika list kosong maka list output adalah [ ]
  - Rekurens: mengambil elemen pertama dari kedua list, menjumlahkan kedua elemen tadi kemudian memasukkannya sebagai elemen pertama dari list output

```
DEFINISI DAN SPESIFIKASI
listPlus : 2 List of integer -> List of integer
{ listPlus(L1, L2) mengembalikan list dengan elemen
 hasil penjumlahan elemen di L1 dan L2.
 prekondisi: L1 dan L2 memiliki jumlah elemen sama }
REALISASI
listPlus(L1, L2) : if isEmpty(L1) then [] { Basis 0 }
                   else { Rekurens }
                     Konso(FirstElmt(L1) + FirstElmt(L2),
                           ListPlus(Tail(L1), Tail(L2)))
```

Kemunculan Nilai Maks

- Fungsi menghasilkan nilai maksimum dan jumlah kemunculan nilai maksimum tsb pd list bilangan integer

  maxNb: List of integer → <integer,integer>
- Contoh: => maxNb([11,3,4,5,11,6,11]) = <11,3>
- Rekursif
  - Basis (Basis 1): List dgn satu elemen e menghasilkan <e,1>
  - Rekurens:

Jika m adalah nilai maksimum dari tail li
dan n adalah jumlah kemunculan m pada tail li
maka ketika memeriksa e, ada 3 kemungkinan:
m < e: nilai maksimum diganti yang baru (m ← e), n=1
m = e: nilai maksimum tetap m, nilai kemunculan n ditambah 1
m > e: nilai maksimum tetap m, nilai kemunculan tetap n

Nilai maks: m; Jumlah kemunculan: n

Fungsi maxNb

DEFINISI DAN SPESIFIKASI

```
maxNb : List of integer --> <integer, integer>
{ maxNb(L) menghasilkan <nilai max, kemunculan nilai max>
 dari suatu list of integer L: <m,n> dengan m adalah nilai
 maksimum di L dan n adalah jumlah kemunculan m dalam L }
```

REALISASI

```
maxNb(L) : if isOneElmt(L) then <FirstElmt(L),1> { Basis }
           else { Rekurens }
                let <m,n> = maxNb(Tail(L)) in
                   depend on m, FirstElmt(L)
                      m < FirstElmt(L): <FirstElmt(L),1>
                      m > FirstElmt(L): <m,n>
                      m = FirstElmt(L): <m,n+1>
```

Latihan 2

Tuliskan definisi, spesifikasi, dan realisasi dari:

a. Fungsi **sumIsiList** menghitung hasil penjumlahan dari seluruh elemen sebuah list of integer l yang tidak kosong.

b. Fungsi **filterGanjil** melakukan filtering terhadap sebuah list of integer li sehingga menghasilkan list dengan elemen yang hanya terdiri atas bilangan ganjil yang muncul di li. Diasumsikan semua elemen li adalah bilangan integer positif atau 0. li mungkin kosong.

List of Character

- **List of Character** adalah list yang elemennya berupa character

Elemen	Character
List of elemen	Teks { List of character }

- Contoh
  - Konso: elemen, List → List *{ untuk list of elemen }*
  - Konso: character, Teks →Teks *{ untuk list of character }*

Hitung A

- **nbA** : Teks -> integer >= 0
- Contoh: => nbA(['b','c','a','d','a','n','a']) = 3
- Rekursif
  - Basis {basis 0}: teks kosong, output: 0
  - Rekurens: Periksa huruf pertama dari teks, jika ‘a’ maka 1 ditambahkan ke nilai kemunculan ‘a’ pada tail, jika bukan ‘a’ maka 0 ditambahkan ke nilai kemunculan ‘a’ pada tail

```
DEFINISI DAN SPESIFIKASI
nbA : Teks -> integer >= 0
{ nbA(Lc) menghasilkan banyaknya kemunculan huruf 'A'
 di teks Lc }
REALISASI
nbA(Lc) : if isEmpty(Lc) then 0  { Basis }
          else { Rekurens }
              nbA(Tail(Lc)) + (if FirstElmt(Lc)=='A’ or
                               FirstElmt(Lc)==’a' then 1 else 0)
```

Latihan 3

Tuliskan definisi, spesifikasi, dan realisasi dari fungsi **isEqFront** menerima masukan 2 buah list of character yang tidak kosong, misalnya T1 dan T2 dan menghasilkan true jika potongan awal list T2 mengandung T1 (dengan panjang dan urutan karakter yang sama). Banyaknya elemen T2 selalu lebih dari atau sama dengan T1.

Contoh:

T1: ['a', 'b', 'c'] T2: ['a', 'b', 'c', 'd', 'e'] Hasil: true
T1: ['a', 'b', 'c'] T2: ['a', 'b', 'c'] Hasil: true
T1: ['a', 'b', 'c'] T2: ['a', 'b', 'a', 'b', 'c', 'd'] Hasil: false
T1: ['a', 'b', 'c'] T2: ['a', 'b', 'd', 'a', 'b', 'c'] Hasil: false

Pada dasarnya semua jenis list dikelola dengan cara yang sama

- Menghitung kemunculan sebuah elemen pada list
  - List of integer
    - `nbXInt : integer, List of integer -> integer`
  - List of character
    - `nbC : character, Teks -> integer`

Menghitung Kemunculan X

- Rekurens
  - Basis: untuk list kosong, kemunculan adalah 0
  - Rekurens: dicek apakah elemen pertama adalah X, jika iya maka nilai 1 ditambahkan pada hasil penghitungan kemunculan X pada tail dari list, jika tidak maka nilai 0 yang ditambahkan

```
nbX(x, L) : if isEmpty(L) then 0  { Basis }
            else  { Rekurens }
               (if (FirstElmt(L) == x) then 1 else 0)
               + nbX(x, Tail(L))
```

List of Elemen

```
nbX(x, L) : if isEmpty(L) then 0   { Basis }
            else  { Rekurens }
                (if (FirstElmt(L) == x) then 1
                                        else 0)
                + nbX(x, Tail(L))
```

List of Integer

```
nbXInt(x, Li) : if isEmpty(Li) then 0   { Basis }
                else  { Rekurens }
                  (if (FirstElmt(Li) == x) then 1
                                           else 0)
                  + nbXInt(x, Tail(Li))
```

List of Character

```
NbC(x, Lc) : if isEmpty(Lc) then 0   { Basis }
             else  { Rekurens }
                 (if (FirstElmt(Lc) == x) then 1
                                          else 0)
                 + nbC(x, Tail(Lc))
```

Latihan 4

Tuliskan realisasi dari fungsi **isOrdered** berikut ini:

```
isOrdered : List of integer -> boolean
{ isOrdered(L) menghasilkan true jika elemen-elemen pada L
   terurut membesar, false jika tidak.
   Prekondisi: L tidak kosong }
```

Latihan 5

Tuliskan realisasi dari fungsi **mergeList** berikut ini:

```
mergeList : 2 List of integer -> List of integer
{ mergeList(L1, L2) menghasilkan list of integer yang
   merupakan hasil penggabungan L1 dan L2, dan tetap
   terurut membesar.
   Prekondisi: L1 dan L2 adalah list terurut membesar
   dan mungkin kosong -}
```

Latihan 6

Tuliskan realisasi dari fungsi **splitList** berikut ini:

```
SplitList : List of integer -> <List of integer,List of integer>
{ splitList(L) menghasilkan 2 list of integer, list
   pertama memuat bilangan positif dan 0 yang merupakan
   elemen dari L  (dengan urutan kemunculan yang tidak
   berubah), sedangkan list kedua memuat bilangan negatif
   elemen L .
   Prekondisi: L mungkin kosong -}
```

Bahan

- Diktat “Dasar Pemrograman, Bag. Pemrograman Fungsional” oleh Inggriani Liem, revisi Februari 2014

Latihan Tambahan

Soal 1

Tuliskan realisasi dari fungsi **isUnique** berikut ini:

```
isUnique : Teks  -> boolean
{ isUnique(Lc) menghasilkan true jika Lc
   adalah list dengan elemen unik, yaitu tidak
   ada elemen pada Lc yang muncul lebih
   dari 1 kali }
```

Soal 2

Tuliskan realisasi dari fungsi **posOfX** berikut ini:

```
posOfX : character, Teks -> integer
{ posOfX(e, Lc) menghasilkan sebuah bilangan integer yang
    menyatakan posisi e pada list of character Lc.
    Jika e bukan elemen dari Lc, fungsi akan menghasilkan 0.
    Prekondisi: Lc memiliki elemen unik }
```

Soal 3

Tuliskan realisasi dari fungsi **splitAlternate** berikut ini:

```
splitAlternate : Teks -> <Teks,Teks>
{ splitAlternate(L)  menghasilkan dua buah list, misalnya L1
   dan L2.
   L1 berisi semua elemen L pada posisi ganjil,
   L2 berisi semua elemen L pada posisi genap -}
```